ICCAD 2026 CAD Contest Problem D
Timing Fixing by Useful Skew

Aaron Wu
Department of Electrical Engineering
National Taiwan University
b13901011@ntu.edu.tw

Yucheng Liu
Department of Electrical Engineering
National Taiwan University
b13901104@ntu.edu.tw

Sheng-Yu Cheng
Department of Electrical Engineering
National Taiwan University
b13901088@ntu.edu.tw

Hong-Bin Lai
Department of Electrical Engineering
National Taiwan University
b13901078@ntu.edu.tw

Abstract

1 Introduction

Timing closure is a critical task in modern VLSI design. As clock frequency increases, setup and hold violations become harder to fix without disturbing the functional logic of the circuit. ICCAD Contest 2026 Problem D focuses on this setting: given an initial clock tree, a buffer library, and data-path delay reports, the goal is to improve timing quality by modifying the clock tree while preserving the original data-path logic.

The key idea behind this problem is useful skew. Instead of treating all clock skew as harmful, useful-skew optimization intentionally changes clock arrival times at flip-flops to improve timing margins. For a setup-critical path, delaying the capture clock can provide more time for data propagation. For a hold-critical path, delaying the launch clock can postpone data launch and reduce the risk of early data arrival. Therefore, by inserting or resizing buffers in the clock tree, the optimizer can improve setup and hold slack without changing combinational logic.

However, this optimization is highly coupled. A clock-tree edit does not affect only one data path; it changes the clock arrival time of an entire downstream subtree. As a result, a move that improves one violating path may degrade other paths that share the same clock-tree nodes. In addition, buffer insertion and upsizing may increase area, while aggressive area recovery may undo previous timing improvements. The objective is therefore not simply to fix the worst violation, but to balance SS setup improvement, FF hold improvement, and clock-tree area.

In this work, we formulate timing fixing by useful skew as a constrained local-search problem over legal clock-tree states. Our solver maintains a mutable clock-tree representation, evaluates timing and area metrics after local edits, and uses reversible trial operations to explore the search space efficiently. Candidate-generation policies decide which clock-tree edits should be evaluated, while acceptance strategies decide how the solver moves from one tree state to another. The experimental discussion focuses on how these policies trade off setup slack, hold slack, and area.

The rest of this report is organized as follows. Section 2 formulates the timing model, objective, and legal action space. Section 3 describes the local-search framework, including candidate generation, action evaluation, acceptance strategies, and legality checks. Section 4 reports experimental results and discusses the trade-offs among the tested optimizers. Section 5 concludes the report and outlines future improvements.

2 Problem Formulation

2.1 Input Design and Legal Solution Space

Each testcase consists of an initial clock tree, a buffer library, and two data-path delay reports under the SS and FF corners. The clock tree contains a clock source as the root, clock buffers as internal nodes, and flip-flops as leaf nodes. The buffer library specifies the available buffer types, including their area, maximum fanout constraint, and fanout-dependent delay under each timing corner. The delay reports give the clock period and the fixed data-path delay between launch and capture flip-flops.

Preprint.

The optimizer is allowed to improve timing only by modifying the clock tree. The data-path delay is fixed, so the combinational logic is not part of the optimization space. A legal output must preserve all original contest nodes and their names, and the relative order among original nodes in the clock tree must remain unchanged. Newly inserted buffers must have unique names, all pins must be connected, each flip-flop must have a single driver, and every buffer must satisfy the fanout limit specified by the buffer library.

We model a solution as a clock-tree state. A state records the cell type of each buffer, the parent-child connectivity of the clock tree, and the set of buffers inserted by the optimizer. The legal solution space consists of all clock trees reachable from the input tree by legal clock-tree edits while satisfying the structural constraints above.

2.2 Clock Arrival Time and Useful Skew

For each flip-flop, the clock arrival time is the total delay from the clock source to that flip-flop through the clock tree. Since buffer delay depends on the selected cell type, the fanout count, and the timing corner, the clock arrival time is also corner-dependent.

A data path connects a launch flip-flop to a capture flip-flop through combinational logic. Let D_{clk}^{launch} and D_{clk}^{capture} denote the clock arrival times from the clock source to the launch and capture flip-flops. We define clock skew as

$$\text{Skew} = D_{clk}^{\text{capture}} - D_{clk}^{\text{launch}}. \tag{1}$$

With this convention, positive skew means that the capture clock arrives later than the launch clock. A clock-tree edit may change the arrival time of an entire downstream subtree, so the skew of many data paths can change after a single local modification.

2.3 Setup/Hold Slack and Timing Metrics

Let D_{data} be the data-path delay, T_{clk} be the clock period, T_{setup} be the setup requirement, and T_{hold} be the hold requirement. With the skew convention above, the setup and hold slacks are

$$\text{Slack}_{\text{setup}} = T_{\text{clk}} - T_{\text{setup}} - D_{\text{data}} + \text{Skew}, \tag{2}$$

$$\text{Slack}_{\text{hold}} = D_{\text{data}} - T_{\text{hold}} - \text{Skew}. \tag{3}$$

A negative setup slack means that data arrives too late at the capture flip-flop. A negative hold slack means that data arrives too early and may overwrite the previous value. Therefore, a setup violation can often be improved by increasing the capture clock delay, while a hold violation can often be improved by increasing the launch clock delay.

Timing quality is measured using total negative slack (TNS) and worst negative slack (WNS). WNS captures the most severe timing violation among all paths, while TNS measures the accumulated amount of negative slack. In this problem, setup timing is evaluated under the setup-oriented SS corner, and hold timing is evaluated under the hold-oriented FF corner.

2.4 Timing-Area Objective and Internal Proxy Score

The objective combines timing and area. Timing quality is measured by TNS and WNS under the SS and FF corners, while area is the total area of the clock-tree buffers. We define the setup score, hold score, and area score as

$$S_{\text{setup}} = \left(1 - \frac{\text{TNS}_{SS}}{\text{TNS}_{SS}^{ori}}\right) + \left(1 - \frac{\text{WNS}_{SS}}{\text{WNS}_{SS}^{ori}}\right), \tag{4}$$

$$S_{\text{hold}} = \left(1 - \frac{\text{TNS}_{FF}}{\text{TNS}_{FF}^{ori}}\right) + \left(1 - \frac{\text{WNS}_{FF}}{\text{WNS}_{FF}^{ori}}\right), \tag{5}$$

$$S_{\text{area}} = 1 - \frac{\text{Area}}{\text{Area}^{ori}}. \tag{6}$$

Each denominator uses the metric value of the original input clock tree. Higher setup and hold scores mean that the negative timing metrics have moved closer to zero, while a higher area score means that the clock-tree buffer area is smaller than the baseline.

The official contest score is a weighted sum of these three components:

$$S_{\text{total}} = \alpha S_{\text{setup}} + \beta S_{\text{hold}} + \gamma S_{\text{area}}. \tag{7}$$

The official weights are not disclosed, so the optimizer cannot directly use the official score as its search objective. According to the Q&A session, the organizers indicated that the weights roughly satisfy $\alpha > \beta \approx \gamma$. Based on this hint, we use a proxy score with fixed weights $\alpha = 0.5$, $\beta = 0.25$, and $\gamma = 0.25$ to guide move selection during optimization. These parameters are not claimed to be the hidden official weights; they are used only as an internal ranking function that reflects the relative importance of setup repair, hold repair, and area control.

This objective captures the main trade-off in the problem. Aggressive buffer insertion or upsizing may improve setup or hold slack, but the same move can reduce the area component and may also hurt timing on other paths. On the other hand, area recovery may remove unnecessary delay but can also undo a previous timing repair. Therefore, the optimizer must balance timing improvement and area cost rather than optimizing only one metric independently.

2.5 Local Edit Semantics

During search, our optimizer explores the solution space with three local edit primitives. First, an *insert* move places a new buffer on an existing clock-tree edge, increasing the clock delay of the downstream subtree. This can adjust useful skew, but also increases area.

Second, a *resize* move changes the cell type of an existing buffer, providing a finer way to increase or decrease clock delay and area without changing the clock-tree topology. Depending on the selected cell, it may improve timing by adding delay or reduce area by using a smaller buffer.

Third, a *remove* move deletes a buffer inserted by the optimizer in an earlier step and reconnects the surrounding tree. This move is used only internally for reversible exploration and area recovery. Original contest nodes remain part of the design and are never removed, so the final output still satisfies the legal modification constraints.

3 Methodology

3.1 Solver Overview

After defining the legal edit operations and the timing-area objective, the remaining question is how to search the large space of legal clock-tree states efficiently. Our solver uses a local-search framework that repeatedly moves from the current clock tree to a nearby legal state. Each iteration is organized around two decisions: which candidate moves should be evaluated, and which evaluated move should be accepted as the next state.

We separate these two decisions into a **candidate-generation policy** and an **acceptance strategy**. The candidate-generation policy defines the local neighborhood visible to the solver in the current iteration. It may focus on violating timing paths, expand around upstream clock-tree regions, or include remove and resize moves for area recovery. The acceptance strategy then decides how the solver moves inside this neighborhood. It may greedily accept the best immediate improvement, probabilistically accept a temporary degradation, or use short-term memory to avoid cycling.

Algorithm 1 summarizes the overall optimization flow. Each iteration first constructs a bounded candidate set, evaluates the candidates through temporary application, and then applies one move according to the acceptance strategy. The solver also maintains the best-seen state separately from the current state, because exploratory strategies may temporarily move to lower-scoring states.

The rest of this section follows the structure of Algorithm 1. Section 3.2 describes the optimization state and the incremental timing evaluation used inside the temporary apply-score-undo loop. Section 3.3 describes how candidate-generation policies construct the move set M. Section 3.4 describes how acceptance strategies choose the move $m^\star$ from the evaluated set. Finally, Section 3.5 explains how the best-seen internal state is converted into a legal output file.

3.2 Optimization State and Incremental Evaluation

The optimizer maintains three main data structures during search. The `ClockTree` is the mutable solution state and stores the active topology and buffer cell types. The `DataPathGraph` stores the fixed timing paths, including the launch flip-flop, capture flip-flop, and data delay of each path. Since useful-skew optimization modifies only the clock tree, this graph remains read-only. The `TimingState` connects the two structures and caches the quantities needed for fast evaluation, including clock arrivals, setup and hold slacks, TNS, WNS, area, and the proxy score.

Candidate evaluation follows the temporary apply-score-undo loop shown in Algorithm 1. For each trial edit, the solver applies the move to the `ClockTree`, updates the affected clock arrivals and related timing paths in `TimingState`, and computes the proxy-score change. The move is then either accepted or rolled back. Rollback restores both the clock-tree state and the timing cache, so rejected candidates do not affect later evaluations. This design avoids recomputing all clock arrivals and slacks for every trial move. Instead, the solver updates only the parts of the timing state affected by the local edit, allowing many candidate moves to be evaluated within the fixed time budget.

3.3 Candidate Generation

Since the legal edit space is too large to evaluate exhaustively, candidate generation defines the local neighborhood seen by the optimizer at each iteration. A good candidate policy should include moves that are likely to improve useful skew, but it should also keep the candidate set small enough that many iterations can be completed within the time budget. We therefore explored several policies with different degrees of timing awareness and search breadth.

The simplest policy is **random generation**. It uniformly samples legal insert, remove, and resize moves from the current clock tree. This policy is the most naive way to explore the solution space, and it does not directly use timing

Algorithm 1 Overall local-search optimization flow

Build the initial `ClockTree`, `DataPathGraph`, and `TimingState`
Evaluate the baseline tree and compute its proxy score
$T_{\text{cur}} \leftarrow$ baseline clock tree
$T_{\text{best}} \leftarrow T_{\text{cur}}$
$S_{\text{best}} \leftarrow S_{\text{proxy}}(T_{\text{cur}})$
while current time $<$ time budget **do**
    Generate a candidate move set M using the candidate policy
    $E \leftarrow \emptyset$
    for all $m \in M$ **do**
        Temporarily apply m to T_{cur}
        Update timing and area metrics
        Compute $\Delta S_{\text{proxy}}(m)$
        Store the evaluated move information in E
        Undo m and restore T_{cur}
    end for
    Select a move $m^{\star}$ from E using the acceptance strategy
    if $m^{\star}$ exists **then**
        Apply $m^{\star}$ to T_{cur}
        Update `TimingState` for the current tree
        if $S_{\text{proxy}}(T_{\text{cur}}) > S_{\text{best}}$ **then**
            $T_{\text{best}} \leftarrow T_{\text{cur}}$
            $S_{\text{best}} \leftarrow S_{\text{proxy}}(T_{\text{cur}})$
        end if
    end if
end while
Restore T_{best}
Perform full timing evaluation
Write `modified_clk_tree.structure`

information to guide move selection. As a result, it may generate many weak moves that do not improve timing. However, it can also expose non-obvious area-recovery or resizing opportunities that more focused policies may miss.

The most direct timing-driven policy is **violation path generation**. It scans all data paths, selects paths with negative slack, and prioritizes the most severe violations. For setup violations, it generates moves near the capture flip-flop; for hold violations, it generates moves near the launch flip-flop. This follows the useful-skew formulas directly, but the resulting neighborhood can be too narrow when the best repair is not close to the violating endpoint.

To address this limitation, the **upstream-window policy** expands the search region from a violating endpoint toward the root of the clock tree. Instead of considering only the endpoint edge, it considers several upstream buffers and edges. These moves may affect a larger downstream subtree and can therefore repair several related timing paths at once.

The **critical-endpoint policy** uses path violations to assign a criticality value to each flip-flop. Negative setup slack contributes to the criticality of the capture flip-flop, while negative hold slack contributes to the criticality of the launch flip-flop. The policy then generates moves around the most critical flip-flops. Compared with selecting individual violating paths, this policy tries to identify endpoints that participate in many violations and may therefore produce moves with broader timing impact.

Finally, the **union-pool policy** combines the action sources of violation path, upstream window, and critical endpoint into one candidate pool. It uses the same total sampling budget as the individual policies, so the goal is not to evaluate more candidates, but to obtain a more diverse candidate set under the same cost. For simulated-annealing-based optimizers, we use a **sampled-union-pool policy**, which samples only one move from this mixed pool in each iteration to match the single-proposal style of simulated annealing.

3.4 Acceptance Strategies

After candidate generation defines the visible neighborhood, the acceptance strategy determines how the search moves through that neighborhood. This decision is important because the proxy score is not a smooth objective. A move that is locally worse may still help the solver reach a better region later, while a purely greedy sequence may quickly become trapped after all immediate improvements are exhausted.

The **greedy strategy** is the most exploitative choice. It evaluates all candidates in the current set, and applies the move with the largest positive proxy-score improvement. This strategy is stable and easy to interpret: every accepted move immediately improves the current solution according to the proxy score. However, its performance depends heavily on the candidate pool. If the pool does not contain a good improving move, greedy search cannot escape the current local optimum.

The **two-step strategy** is a greedy strategy with two objective phases. The first phase greedily accepts moves that improve a slack-focused objective, with stronger emphasis on setup TNS and WNS. The second phase switches back to the full proxy score, so setup, hold, and area are optimized together. This design tries to create timing margin first and then recover overall score, but Its weakness is that area recovery may undo previous timing repairs, and timing-focused moves may create too much area overhead for the second phase to repair.

Simulated annealing[1] introduces stochastic exploration. At each step, it samples a proposal, evaluates its proxy-score change, and always accepts improving moves. Different from greedy search, a worsening move may also be accepted with a temperature-controlled probability. At high temperature, the search can move through worse intermediate states and escape local optima; at low temperature, it behaves more like greedy search. The solver still records the best-seen state separately, so temporary degradation does not directly determine the final output.

Iterated simulated annealing repeats this exploration process in several rounds. Between rounds, the solver restores the best-seen state and performs small greedy cleanup batches. The SA phases provide exploration, while the cleanup phases exploit local improvements around promising regions. This design attempts to keep the ability to escape local optima without letting the current state drift too far from a high-quality solution.

Tabu search [2] uses short-term memory instead of random temperature to guide exploration. At each step, it evaluates the candidate set and chooses the best non-tabu move. Recently used moves are marked tabu so that the solver is discouraged from cycling between the same few states. A tabu move can still be accepted by aspiration if it improves the global best score. This makes tabu search more exploratory than greedy search while still keeping each step strongly score-driven.

Overall, candidate generation and acceptance play complementary roles. Candidate policies decide which legal edits are worth looking at, while acceptance strategies decide how aggressively the solver should exploit or explore the evaluated moves. The optimizer variants evaluated in Section 4 are different combinations of these two choices.

3.5 Output Format Validation

Output validity is enforced during move construction rather than repaired after optimization. Inserted buffers receive unique generated names, resize and insertion moves are rejected if the selected cell type violates the fanout limit, and remove moves are limited to optimizer-inserted buffers. Original contest nodes remain active and keep their names, while removed search-only buffers are marked inactive so that they cannot appear in the final output.

After the optimizer finishes, the solver restores the best-seen `ClockTree` and writes only active nodes through a preorder traversal with depth information, matching the input clock-tree format. Since the writer emits only active parent-child relationships, each flip-flop keeps a single clock-tree parent, the tree remains well formed, and every emitted buffer satisfies the buffer-library fanout table. The final file is written through a temporary file followed by a rename, so the output path is replaced only after a complete `modified_clk_tree.structure` file has been produced.

4 Experimental Results and Discussion

4.1 Experimental Setup and Metrics

We evaluate all optimizer variants on five public testcases. Each optimizer is run with five different random seeds, resulting in a total of 325 runs. The reported score is the proxy score defined in Section 2, using fixed weights for setup timing, hold timing, and area. During each run, we record detailed progress traces in order to observe its trajectory of solving the problem. These traces include the elapsed time, the best score found so far, the current timing metrics under both SS and FF corners, and the current clock-tree area. The traces are then plotted into its trajectory over time. Their purpose is to show how each optimizer behaves: how quickly it improves the solution and how the improvements are distributed across setup timing, hold timing and area.

Each trajectory figure contains four panels. The first panel shows the best-score gap over time. To make trajectories from different testcases comparable, we align each testcase to its own best observed score instead of plotting raw scores directly. For each testcase, we first identify the best score that was ever observed across all optimizers, seeds, and time points in our experiments. Then, for each run, we measure how far its current best score is from this reference value. Because this reference is the best observed score, the gap is always zero or negative. A curve that is closer to zero means that the optimizer is closer to the best result we have seen for that testcase. It is important to note that this reference is based only on our experiments and does not represent a true optimal solution.

The other three panels break down the optimization process into setup timing, hold timing, and area. For each run, we treat the first recorded state as the baseline. We then measure how much each quantity improves relative to this starting point. For setup and hold timing, we consider both total negative slack and worst negative slack, and combine their improvements into a single score. For area, we measure how much the clock-tree area is reduced compared to the initial value.

To produce a single curve for each optimizer, we aggregate results across runs in several steps. First, we divide the total runtime into uniform time intervals. For each run, we record the latest available value within each interval, and if a

value is missing, we carry forward the most recent one. Then, for each testcase, we combine the five seeds by taking their median value at each time point. Finally, we average these testcase-level medians to obtain the main curve. The shaded region around the curve shows the variation across different testcases, which means the shaded area reflects how performance differs between testcases, rather than uncertainty due to random seeds.

4.2 Effect of Different Candidate Generation Policies

We first isolate the effect of different candidate generation policies by fixing the acceptance policy to greedy best-score selection. Under this setting, the optimizer can only accept an immediately improving move, so the result mainly reflects whether the candidate pool contains useful local edits.

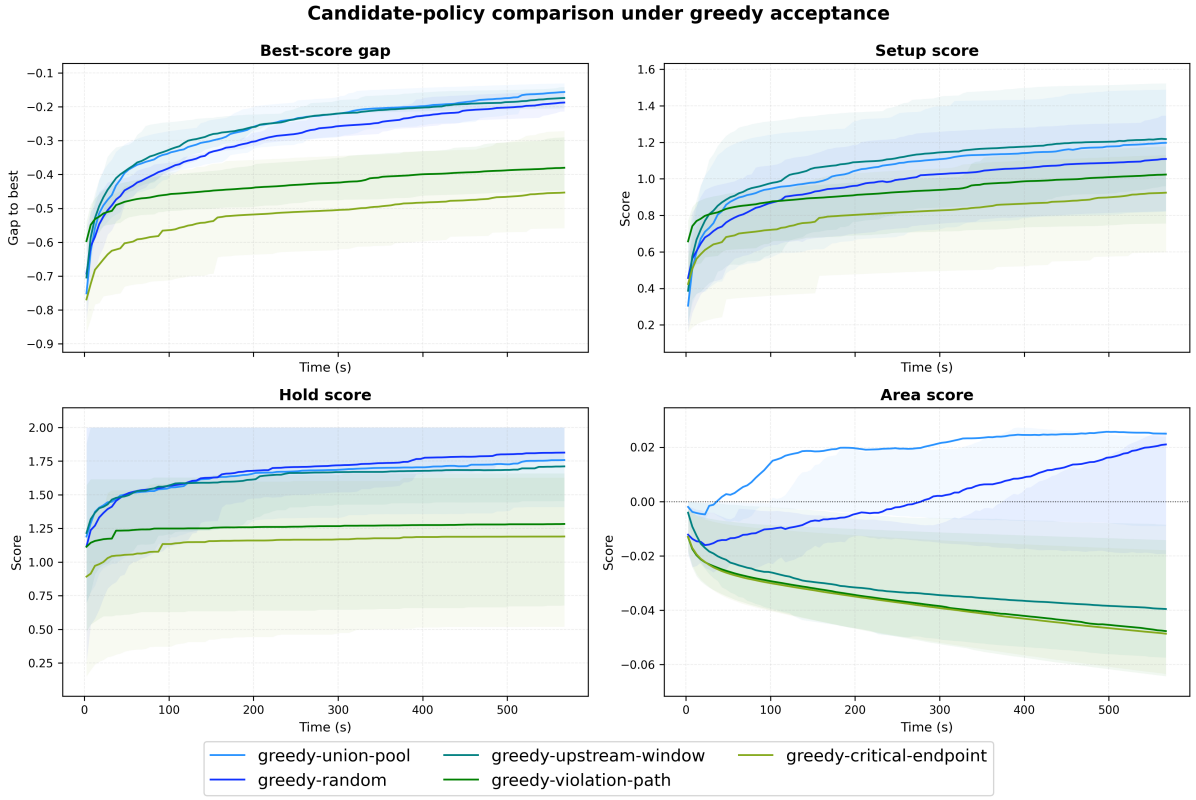

Figure 1: Candidate-policy comparison under greedy acceptance. Each method uses the same greedy acceptance rule, but differs in how candidate clock-tree edits are generated. The panels show the best-score gap, setup score, hold score, and area score over the fixed time budget.

Figure 1 shows that the union-pool and upstream window candidate policies reach the best final best-score gap among the greedy variants, while random remains competitive. The violation-path and critical-endpoint policies improve timing early, but their curves flatten at a worse gap. This supports the claim that endpoint-focused candidate generation is too narrow for useful-skew optimization, which a clock-tree edit affects an entire downstream subtree, and a good repair is often not located directly at the currently violating endpoint. The area panel also explains why pure timing awareness is not enough. Some focused policies improve setup or hold but drift into negative area score, whereas the more diverse union pool can expose insert, remove, and resize moves that preserve or recover area while still improving timing. Therefore, greedy search benefits more from candidate diversity than from a narrowly targeted violation-only neighborhood.

4.3 Effect of Different Acceptance Policies

We next fix the candidate family and compare how different acceptance policies move through the generated neighborhood. This experiment separates the quality of candidate generation from the ability of the search strategy to escape local optima.

4.3.1 Random Candidate Policy

With random candidates, the optimizer sees a broad but noisy neighborhood. This setting is useful for testing whether an acceptance policy can extract useful moves from an unstructured candidate stream.

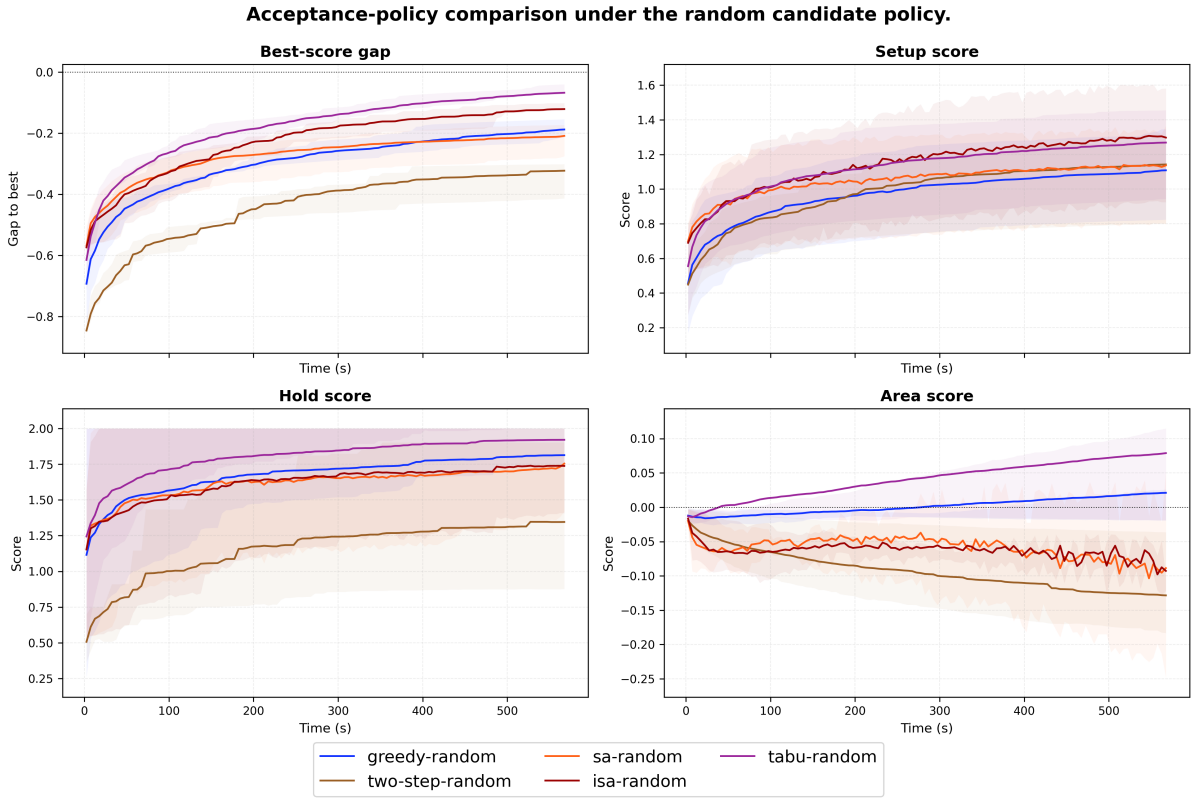

Figure 2: Acceptance-policy comparison under the random candidate policy. The candidate generator is fixed to random sampling, while the acceptance rule varies among greedy, two-step, SA, ISA, and tabu search.

Figure 2 shows that tabu search obtains the closest best-score gap to zero under random candidates. The reason is visible in the component plots: tabu-random is not merely strong in one metric, but maintains a balanced trajectory. It obtains high hold score, competitive setup score, and a positive area score by the end of the run. SA and ISA also escape the limitations of greedy search and improve timing, but their area curves remain negative, indicating that part of their timing gain is paid for by extra buffer area. The two-step method is the weakest in this setting. Its timing-first phase does not create enough durable timing advantage, and the later cleanup phase does not recover enough area or hold quality to close the gap. The main insight is that a random neighborhood can be useful, but only if the acceptance strategy has a mechanism, such as tabu memory, to avoid short cycles while still selecting high-quality moves.

4.3.2 Union-Based Candidate Policies

We also compare acceptance policies when the candidate pool is built from the union of timing-aware sources. For SA and ISA, the sampled-union-pool variant uses a single sampled proposal per iteration to match the single-proposal style of annealing.

Figure 3 shows a different trade-off from the random candidate experiment. *SA-sampled-union-pool* and *ISA-sampled-union-pool* improve setup aggressively and also obtain strong hold scores, but both suffer from a large negative area score, especially SA. This indicates that sampled union proposals are very effective at finding timing-repair moves, but the stochastic acceptance process tends to keep many area-increasing edits. *Tabu-union-pool* is more conservative: its setup score is lower than the annealing-based methods, but its area score stays near the baseline. *Two-step-union-pool* again remains weak because the fixed timing-repair-then-recovery schedule is too rigid for the coupled setup-hold-area objective. Overall, this figure shows that a richer candidate pool alone does not guarantee a better final score; the acceptance rule must also control area while exploiting the timing opportunities in the pool.

4.4 ISA and Tabu as Search-Memory Strategies

The previous experiments suggest that the strongest strategies are not the ones that maximize a single component. We therefore compare ISA and tabu search more directly, since both methods use search history to avoid the limitations of plain greedy search. ISA uses repeated annealing rounds with best-state restoration, while tabu search uses a short-term tabu list and aspiration to avoid cycling while still accepting globally promising moves.

Figure 4 explains why *tabu-random* becomes the strongest variant in the final ranking. *ISA-sampled-union-pool* has the highest setup trajectory, but its area score decreases sharply and remains strongly negative. This means that its setup repair is obtained by keeping many area-expensive clock-tree edits. *Tabu-union-pool* avoids this severe area collapse,

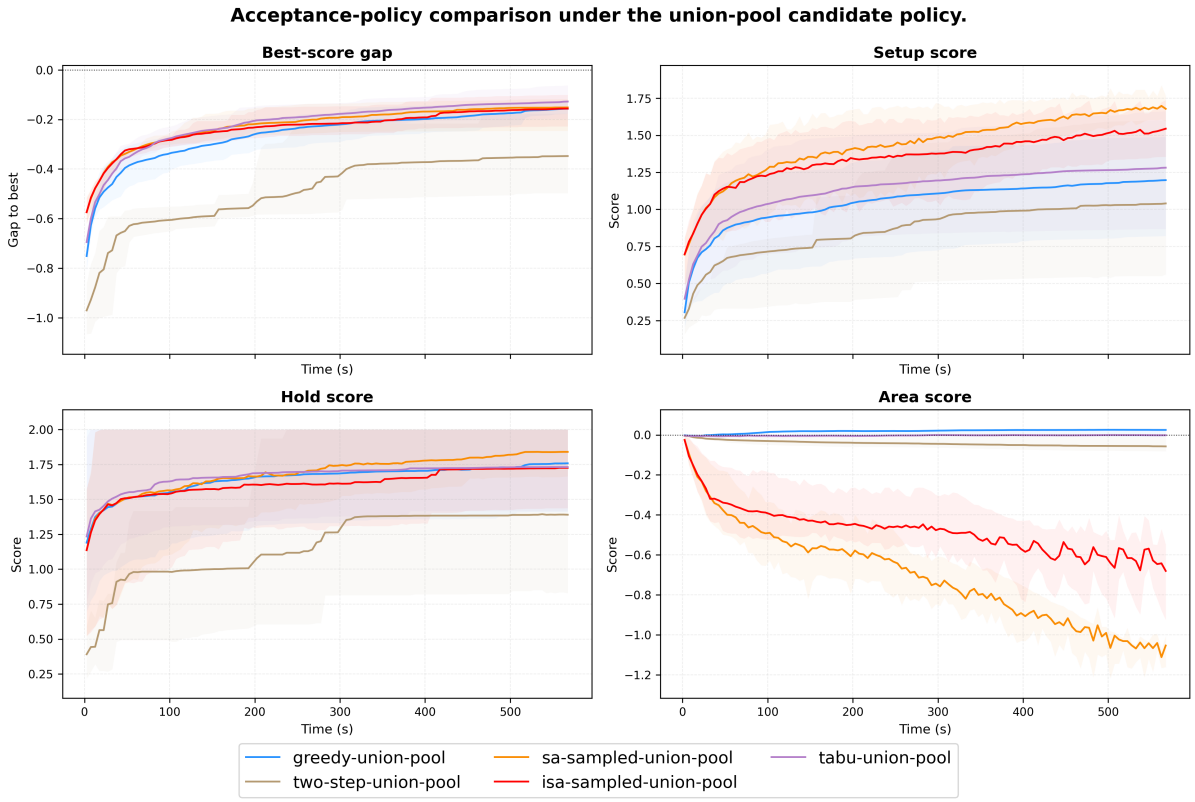

Figure 3: Acceptance-policy comparison under union-based candidate policies. Greedy, two-step, and tabu use the union pool, while SA and ISA use sampled union-pool proposals.

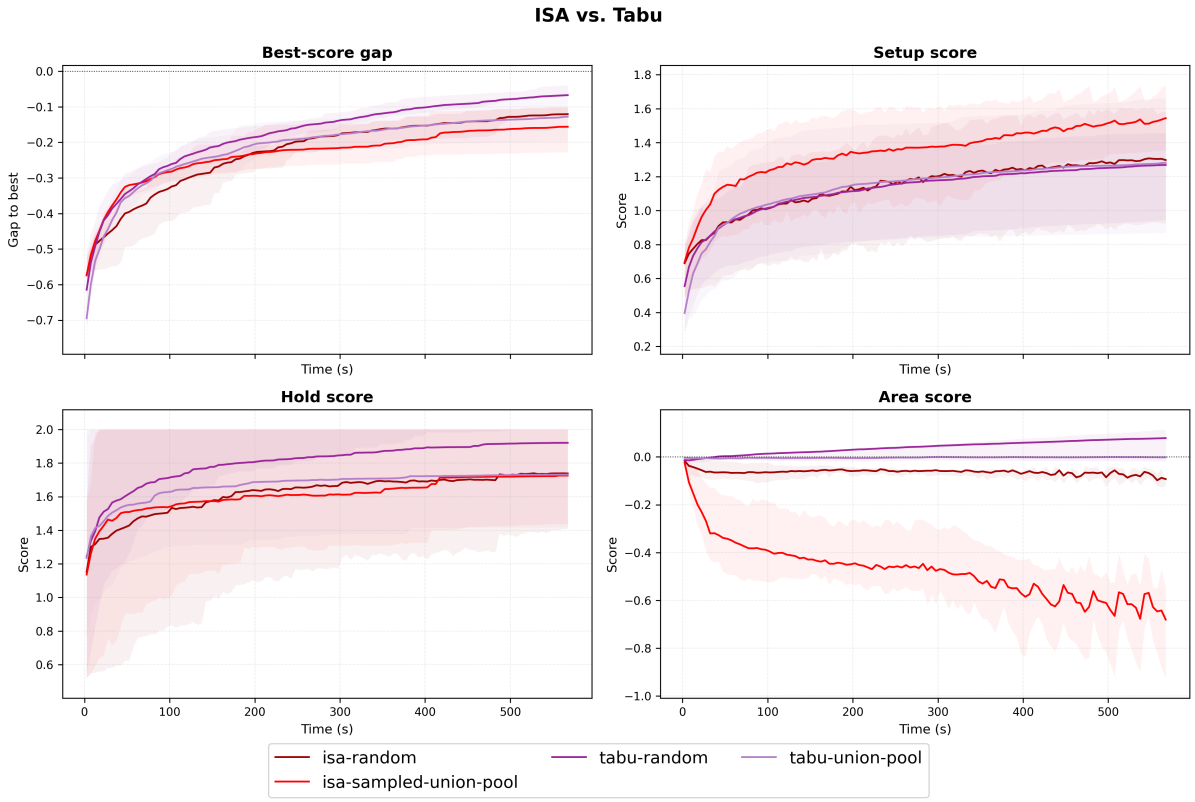

Figure 4: Direct comparison between ISA and tabu variants. The figure compares random and union-based variants of ISA and tabu search using the same four trajectory components.

but its timing components are not as strong as those of *tabu-random*. In contrast, *tabu-random* gives the best overall balance: it approaches the empirical best-score gap most closely, achieves the strongest hold trajectory among these variants, and keeps the area score positive.

This comparison also clarifies the role of random candidate generation. Random proposals are not better because they are more targeted. Instead, they expose a broader set of non-obvious insert, remove, and resize opportunities that a violation-focused union pool may miss. Tabu memory is then strong enough to extract useful moves from this broader neighborhood without repeatedly undoing recent moves. This explains why *tabu-random* outperforms *tabu-union-pool*: the random neighborhood is less biased toward immediate violation repair, while tabu search keeps the trajectory score-driven and prevents short cycles.

Overall, *tabu-random* achieves the highest average score because it does not optimize setup, hold, or area in isolation. It does not have the largest setup component, but it obtains the best hold component and also reduces area. In contrast, *sa-sampled-union-pool* has the highest setup component, but its strongly negative area score pulls down its final score.

4.5 Overall Optimizer Comparison

Table 1 summarizes the final score decomposition for all optimizer variants. This table combines the ablations above and shows which policy combination gives the best overall timing-area trade-off.

Table 1: Performance comparison of different optimizers. Standard deviation is computed from the final scores over five testcases and five seeds.

Optimizer	Avg. Score	Setup	Hold	Area	Std.
tabu-random	1.140 722	1.276 422	1.916 255	0.093 788	0.235 526
isa-random	1.084 773	1.322 733	1.737 303	−0.043 676	0.273 375
tabu-union-pool	1.062 058	1.259 221	1.731 261	−0.001 474	0.274 410
isa-sampled-union-pool	1.044 759	1.521 136	1.721 173	−0.584 406	0.183 581
sa-sampled-union-pool	1.044 015	1.653 149	1.833 949	−0.964 191	0.165 838
greedy-union-pool	1.042 167	1.193 238	1.757 868	0.024 327	0.266 567
greedy-upstream-window	1.031 489	1.227 970	1.710 767	−0.040 751	0.275 511
greedy-random	1.009 059	1.104 560	1.806 695	0.020 420	0.214 807
sa-random	0.998 808	1.149 864	1.750 100	−0.054 596	0.276 991
two-step-random	0.876 061	1.148 869	1.334 986	−0.128 479	0.318 905
two-step-union-pool	0.859 649	1.045 729	1.405 060	−0.057 920	0.401 216
greedy-violation-path	0.829 324	1.032 678	1.300 464	−0.048 521	0.277 805
greedy-critical-endpoint	0.746 583	0.923 081	1.188 662	−0.048 491	0.375 666

5 Conclusion

This work formulates useful-skew timing fixing as a constrained local-search problem over legal clock-tree states. The solver modifies only the clock tree through buffer insertion, inserted-buffer removal, and buffer resizing, and uses a proxy score to balance SS setup repair, FF hold repair, and area control. Internally, a mutable `ClockTree`, a fixed `DataPathGraph`, and an incremental `TimingState` support reversible candidate evaluation, so different search policies can be compared while keeping the final output legal.

The experimental results show that both candidate generation and acceptance strategy are decisive. Greedy search is stable and benefits from the diverse union-pool neighborhood, but it cannot pass through temporary score degradation. SA and ISA explore more aggressively and achieve strong setup improvement with sampled-union proposals, but their area penalty reduces the final score. Two-step optimization captures the intended timing-first and cleanup structure, yet the fixed phase boundary is too rigid for the coupled setup-hold-area objective.

Among all tested optimizers, *tabu-random* achieves the best average score. Its advantage comes from combining a broad random neighborhood with score-driven tabu memory: random candidates expose non-local insert, remove, and resize opportunities, while tabu search avoids short cycles and keeps the current trajectory close to high-quality states. As a result, it does not maximize setup improvement alone, but obtains the best balance among setup repair, hold repair, and area reduction. Future work could improve this framework by adapting the candidate policy to the current violation pattern, calibrating the proxy weights with more feedback, and adding a stronger late-stage area-aware resize/remove polish.

Author Contributions

Hong Bin Lai did nothing :) That's True, I'm very sorry.I find the branch yesterday but I see the first version submission when I'm writing.I know it's a bad excuse, and I really doesn't do anything.

Generative AI Usage Disclosure

We used generative AI tools during the development of this project. In particular, OpenAI Codex and Cursor was used extensively to assist with source-code implementation, including parser construction, clock-tree data structures, timing evaluation routines, optimization-loop implementation, test scripts, and refactoring. Approximately 80% of the submitted source code was initially generated or modified with the assistance of Codex or Cursor.

The algorithmic ideas, including the useful-skew optimization strategy, candidate generation policies, acceptance strategies, timing-area trade-off considerations, experimental design, and final optimizer selection, were designed and evaluated by the team members. All AI-generated code was manually reviewed, integrated, debugged, and tested by the team. The reported experimental results were obtained by executing our implemented solver on the test cases; no experimental data or performance numbers were generated by AI tools.

We also used ChatGPT/Codex to assist with report organization, wording refinement, and documentation polishing. The final technical content, claims, results, and conclusions were checked and approved by the team.

References

[1] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. Optimization by simulated annealing. *science*, 220(4598):671–680, 1983.

[2] Fred Glover. Tabu searchpart i. *ORSA Journal on computing*, 1(3):190–206, 1989.